

Lecture 14

Abstract Base Classes

Abstract classes Concrete Classes

- **Abstract Classes**

- Classes from which it is never *intended to instantiate* any objects, e.g., shape class
 - **Incomplete**—derived classes must define the “missing pieces”.
 - Too generic to define real objects.
- Normally used as base classes
 - Provide appropriate base class frameworks from which other classes can inherit.

- **Concrete Classes**

- Classes used to *instantiate objects*
- Must provide implementation for *every* member function they define
- Until now, all our class examples have been concrete, shape, triangle, Animal, Student etc.

Abstract Base Classes

- An abstract class is a class that can only be derived from; no objects can be instantiated it.
- Its purpose is to define an interface and provide a common base class for derived classes.
- A base class becomes an abstract class by declaring a **virtual function pure**:
`virtual void Draw() = 0;`
- Derived classes must implement all pure virtual functions. If a derived class does not implement these functions, then it becomes an abstract class as well.
- Abstract classes are not required to implement their pure virtual functions.

Pure virtual Functions

- A class is made *abstract* by declaring one or more of its virtual functions to be “pure”
 - By placing `"= 0"` in its declaration

- Example

```
virtual void draw() = 0;
```

- `"= 0"` is known as a *pure specifier*.
- Tells compiler that there *is no* implementation.

Pure virtual Functions (continued)

- Every *concrete* derived class, e.g., class Triangle, must override all base-class **pure virtual functions**
 - with concrete implementations
- If even one *pure virtual function* is not overridden, the derived-class will also be *abstract*
 - Compiler will refuse to create any objects of the class

Instantiating ABC

```
class A {  
public:  
    virtual void x() = 0;  
    virtual void y() = 0;  
};
```

```
class B : public A {  
public:  
    virtual void x();  
};
```

```
class C : public B {  
public:  
    virtual void y();  
};
```

```
int main () {  
    A a; //Error  
    B b; //Error  
    C c; /OKay  
    return 0;  
};
```

- A is an abstract (base) class
 - Declares pure virtual functions (=0)
 - May also have non-virtual methods, as well as virtual methods that are not pure virtual
- Derived classes override pure virtual methods (and thus become concrete)
 - B overrides **x()**, C overrides **y()**
- Can't instantiate an abstract class
 - class that declares pure virtual functions
 - or inherits ones that are not overridden
 - A and B are abstract, C is concrete class

Pure Virtual Functions in C++

```
class A {  
public:  
    virtual void x() = 0;  
    virtual void y() = 0;  
};
```

```
class B : public A {  
public:  
    virtual void x();  
};
```

```
class C : public B {  
public:  
    virtual void y();  
};
```

```
int main () {  
    A * ap = new C;  
    ap->x ();  
    ap->y ();  
    delete ap;  
    return 0;  
};
```

- A is an abstract (base) class
 - Declares pure virtual functions (=0)
 - May also have non-virtual methods, as well as virtual methods that are not pure virtual
- Derived classes override pure virtual methods (and thus become concrete)
 - B overrides **x()**, C overrides **y()**
- Can't instantiate an abstract class
 - class that declares pure virtual functions
 - or inherits ones that are not overridden
 - A and B are abstract, C is concrete class
- Can still have a pointer or reference to an abstract class type
 - Useful for polymorphism

Instantiating Abstract base classes

- An ABC cannot be used as an argument type in a call by value.
- Since they cannot be instantiated, passing them by value (which requires creating a copy) is impossible.

```
class A {
public:
    virtual void x() = 0;
    virtual void y() = 0;
};

class B : public A {
public:
    virtual void x();
    virtual void y();
};

// ERROR: Cannot instantiate ABC
void func(A a) { a.x(); }

int main () {
    C c;
    func(c);
    return 0;
};
```


Instantiating Abstract base classes

- An ABC cannot be used as a return type (if return by value)
- Since they cannot be instantiated, passing them by value (which requires creating a copy) is impossible.
- Returning by value requires creating a copy of the object. Since ABCs are incomplete (due to pure virtual functions), the compiler cannot instantiate or copy them. Only references or pointers to ABCs can be returned, pointing to a concrete derived object.

```
class A {
public:
    virtual void x() = 0;
    virtual void y() = 0;
};
class B : public A {
public:
    virtual void x();
    virtual void y();
};
// ERROR: Cannot instantiate ABC
A func() { }

int main () {

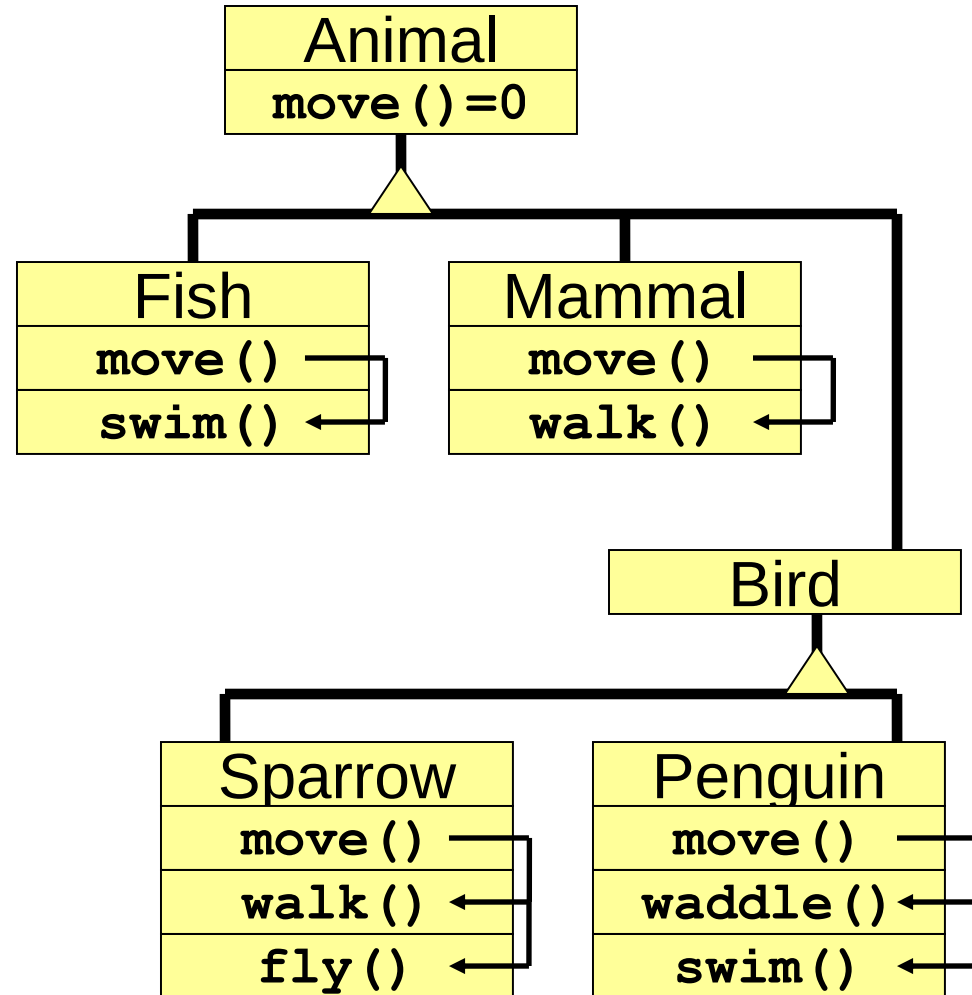
A a = func();
    return 0;
};
```

Purpose of ABC

- Although a program can't make an object from an abstract class, there are some things that an abstract class can still do:
 - be a superclass
 - have concrete features (both variables and functions) that can be inherited by subclasses
 - participate in (i.e., be the target of) an upcast
 - participate in polymorphism

Design with Pure Virtual Functions

- Pure virtual functions let us specify interfaces appropriately
 - But let us defer implementation decisions until later (subclasses)
- As the type hierarchy is extended, pure virtual functions are replaced
 - By virtual functions that fill in (and may override) the implementation details
 - Key idea: *refinement*



Example: Revisiting Shape Class

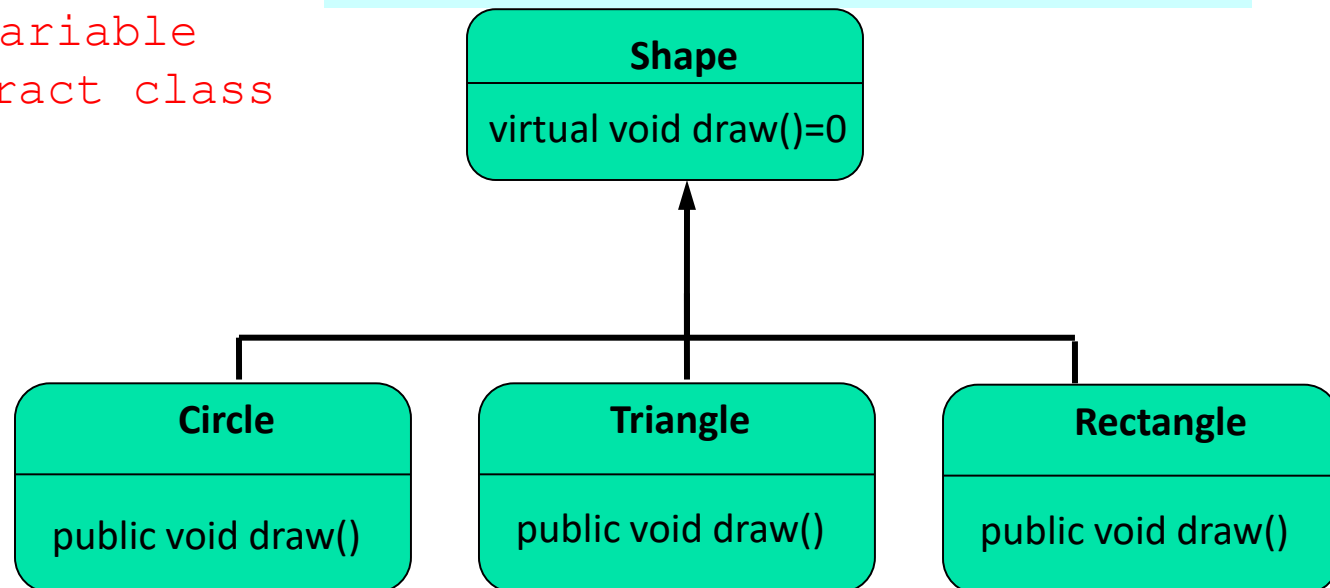
- Some classes exist logically but not physically.
- Example : Shape
 - Shape makes sense only as a base of some classes derived from it. Serves as a “category”
 - Hence instantiation of such a class must be prevented

```
class Shape    //Abstract
{
    public :
    //Pure virtual Function
    virtual void draw() = 0;
}
```

A class with one or more pure virtual functions is an **Abstract Class**

Objects of abstract class can't be created

```
Shape s; //error: variable
        //of an abstract class
```



Example: Revisiting Shape Class (cont'd)

- A pure virtual function not defined in the derived class remains a pure virtual function.
- Hence derived class also becomes abstract

```
class Shape    //Abstract
{
    public :
    //Pure virtual Function
    virtual void draw() = 0;
}
```

```
class Circle : public Shape { //No draw() - Abstract
    public :
    void print() {
        cout << "I am a circle" << endl;
    }
}
class Rectangle : public Shape {
    public :
    void draw() { // Override Shape::draw()
        cout << "Drawing Rectangle" << endl;
    }
}
```

```
Rectangle r; // Valid
```

```
Circle c; // error : variable of an abstract class
```

Pure Virtual functions

- It is still possible to provide definition of a pure virtual function in the base class
- The class still remains abstract and functions must be redefined in the derived classes, but a common piece of code can be kept there to facilitate reuse
- In this case, they can not be declared **inline**

```
class Shape { //Abstract
public :
    virtual void draw() = 0;
};

// OK, not defined inline
void Shape::draw() {
    cout << "Shape" << endl;
}
```

```
class Rectangle : public Shape
{
    public :
        void draw() {
            Shape::draw(); //Reuse
            cout << "Rectangle" << endl;
        }
}
```